Software Documentation

1. Software Overview

- The software controls the car. Via the app on MIT App Inventor, the user can control the care to move manually using the arrow navigators or selecting a destined path or on auto mode – where the car detects obstacles, avoids it, and keeps moving forward.
- Softwares used : Micropython and MIT App Inventor

2. System Logic

1. ESP32 powers on and MicroPython runs main.py automatically
2. Bluetooth begins advertising as "ESP32-NDtV"
3. press scan button on the app
4. device searches for Bluetooth
5. connect to ESP32-NDtV
6. on manual mode :
    a. auto mode is on false
    b. touch down on Up arrow selects “F” on the list of the code which initializes all 4 motors to spin forward at speed 1000
    c. touching down Down arrow selects “B” on the list of the code which initializes all 4 motors spin backward at speed 1000
    d. touching down Left arrow selects “L” on the list of the code which moves Left motors reverse, right motors forward for 0.2s
    e. touching down Right arrow selects “R” on the list of the code which moves Left motors reverse, right motors forward for 0.2s
7. by selecting defined paths :
    a. selecting Mumbai to Dehli runs path_md() —2m straight
    b. selecting Mumbai to Panjim runs path_mp() —1m straight
    c. selecting Panjim to Dehli runs path_pd() — 1 m straight, 1m left, 1 m straight
    d. selecting Mumbai to Tezpur runs path_mt() — 1m straight, 1m right, 1 m straight
8. by selecting auto mode :
    a. Sets auto_mode = True, enters autonomous mode
    b. Ultrasonic sensor reads distance every loop cycle
    c. If distance is greater than 10cm: move forward, NeoPixel glows green
    d. If distance is 10cm or less: call avoid_obstacle()

e. avoid_obstacle() - stops, reverses slightly, scans left and right with servo
f. Compares left and right distances, turns toward the clearer side
g. Moves forward briefly to clear the obstacle, then resumes
h. If any manual command is received: stop and return to manual mode

3. Flowchart

Power ON — ESP32 boots

Pins, PWM, NeoPixel, Servo initialised

Bluetooth advertising starts (ESP32-NDtV)

Waiting for phone to connect via BLE

Phone connects — main while loop begins

MANUAL MODE

(auto_mode = False)

Receive BT command (F/B/L/R/S)

▼ Execute motor function

▼ Reset value to empty

▼ If 'auto' received → switch mode

AUTO MODE

(auto_mode = True)

Read distance sensor

▼ If dist ≤ 10cm → avoid_obstacle()

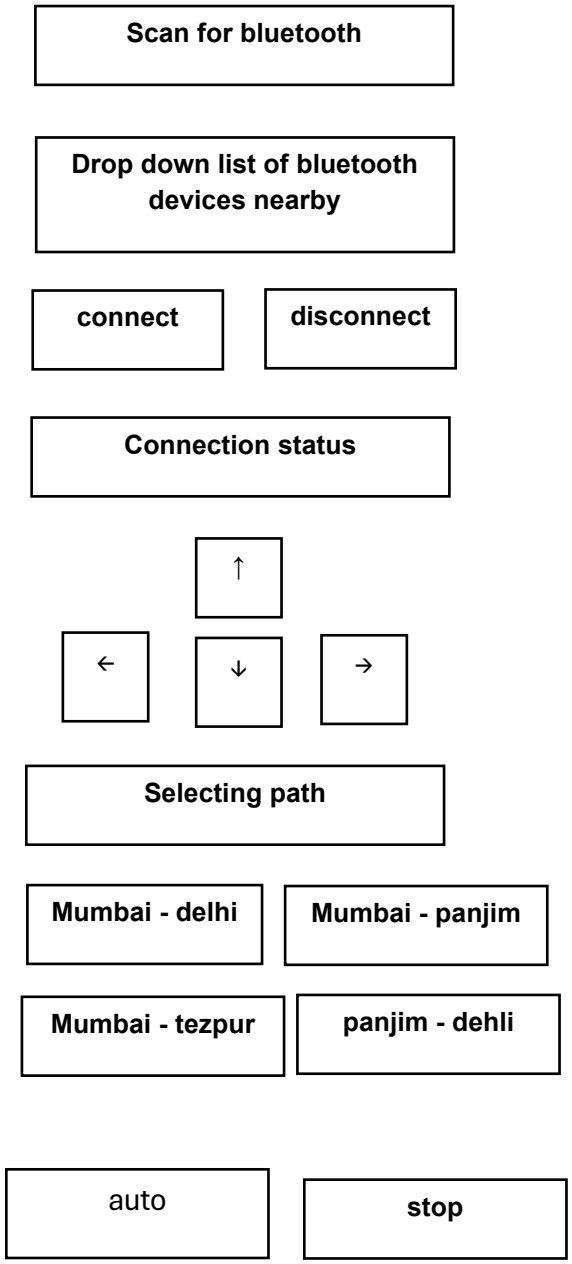
▼ Else → forward() + neo_running()

▼ If manual cmd → switch back

Loop repeats every 30ms (time.sleep_ms(30))

4. App Flow (If MIT App Inventor)

- Screen flow



- Button actions
    - Scan – scans for nearby bluetooth signals
    - Connect – connects to the selected bluetooth
    - Disconnect – disconnects from the Bluetooth
    - Up arrow – moves forward
    - Left arrow – moves to left
    - Right arrow - moves to right
    - Down arrow - moves backwards
    - Mumbai to Dehli - 2m straight
    - Mumbai to Panjim - 1m straight
    - Panjim to Dehli - 1 m straight, 1m left, 1 m straight
    - Mumbai to Tezpur - 1m straight, 1m right, 1 m straight
    - Auto – auto obstacle detection mode
    - Stop – stops all movement

- Data flow
    - User taps a button on the MIT App Inventor app (e.g. the up arrow)
    - App triggers a TouchDown/click event on that button
    - App calls BluetoothLE1.WriteBytes with the command string (e.g. "F")
    - Command is sent wirelessly over BLE to the ESP32's characteristic UUID
    - The event_occured() function is called
    - variable "value" is updated to the received string (e.g. value = "F")
    - The main while loop picks up the new value on its next loop (every 30ms)
    - The matching if/elif condition executes the correct function (e.g. forward())
    - Motor pins and PWM speeds are set — wheels start spinning
    - value is reset back to empty string, ready for the next command